

ユーティリティ (MidiNoteUtility / MidiMessageBuilder)

このページでは、MIDI 1.0 の送受信および関連ヘルパーのランタイムユーティリティについて説明します。

名前空間: `jp.kshoji.unity.midi.util`

配置:

- `Assets/MIDI/Scripts/Utilities/MusicTheory/MidiNoteUtility.cs`
- `Assets/MIDI/Scripts/Utilities/Messaging/MidiMessageBuilder.cs`
- `Assets/MIDI/Scripts/Utilities/Messaging/PitchBendUtility.cs`
- `Assets/MIDI/Scripts/Utilities/Messaging/MidiControlSmoothing.cs`
- `Assets/MIDI/Scripts/Utilities/Messaging/AutomationInterpolation.cs`
- `Assets/MIDI/Scripts/Utilities/Smf/` — `TempoMapExtractor` , `MeasureTimeUtility` , `TupletUtility` , `SwingUtility` , `MidiMetaMessageFactory` (詳細は [SMF ツール](#))
- `Assets/MIDI/Scripts/Editor/UI/PianoKeyboardElement.cs` (`MidiKeyboardLogic`)

MidiNoteUtility

MIDI ノート番号と音名の相互変換、オクターブ計算、半音トランスポーズなどを提供する静的ユーティリティです。
Unity や MIDI プラグインに依存しない pure C# 実装です。

主な定数

定数	値	説明
MiddleC	60	General MIDI における C4
MinNote	0	最小ノート番号
MaxNote	127	最大ノート番号

API 一覧

メソッド	説明	例
ToNoteName(noteNumber, useSharps)	ノート番号を音名に変換	60 → "C4"
TryParseNoteName(name, out noteNumber)	音名をノート番号に変換	"C#4" → 61
GetOctave(noteNumber)	オクターブ番号を取得	60 → 4
GetPitchClass(noteNumber)	ピッチクラス (0-11) を取得	64 → 4 (E)
Transpose(noteNumber, semitones)	半音トランスポーズ (0-127 にクランプ)	(60, 12) → 72
FromPitchClassAndOctave(pitchClass, octave)	ピッチクラスとオクターブからノート番号を算出	(0, 4) → 60
FormatNoteWithNumber(noteNumber)	音名と番号を併記	60 → "C4 (60)"

使用例

```
using jp.kshoji.unity.midi.util;

// ノート番号 → 音名
Debug.Log(MidiNoteUtility.ToNoteName(60));           // "C4"
Debug.Log(MidiNoteUtility.ToNoteName(61, useSharps: false)); // "Db4"

// 音名 → ノート番号
if (MidiNoteUtility.TryParseNoteName("C#4", out var note))
    Debug.Log(note); // 61

// トランスポーズ
var transposed = MidiNoteUtility.Transpose(60, 7);    // G4 (67)
```

オクターブ規約

このユーティリティは MIDI 標準のオクターブ表記を使用します。

```
octave = (noteNumber / 12) - 1
```

例: ノート番号 60 → C4、0 → C-1、127 → G9

MidiMessageBuilder (MidiSend)

MidiManager の送信 API を Fluent インターフェースでラップし、チェーン可能な MIDI メッセージ送信を提供します。
内部では MidiManager.Instance.SendMidi* を呼び出します。

エントリポイント

メソッド	説明
MidiSend.To(deviceId)	指定デバイスへのビルダーを取得
MidiSend.ToFirstOutput()	最初出力デバイスへのビルダーを取得

deviceId が空の場合は ArgumentException、出力デバイスが存在しない場合は InvalidOperationException がスローされます。

ビルダーメソッド

メソッド	説明
Group(int)	MIDI 2.0 グループ (0-15) を設定 (デフォルト 0)
Channel(int)	MIDI チャンネル (0-15) を設定 (デフォルト 0)
NoteOn(note, velocity)	Note On を送信 (velocity デフォルト 127)
NoteOff(note, velocity)	Note Off を送信 (velocity デフォルト 0)
ControlChange(controller, value)	Control Change を送信
ProgramChange(program)	Program Change を送信
PitchWheel(amount)	ピッチベンド送信 (14-bit、0-16383、センター 8192)
PitchWheelNormalized(value)	正規化 float からピッチベンド送信 (PitchBendUtility 経由)
SystemExclusive(byte[])	SysEx 送信
AllNotesOff() / AllNotesOff(channel)	All Notes Off (CC 123) を全チャンネルまたは指定チャンネルへ

各メソッドは this を返すため、チェーン呼び出しが可能です。

使用例

```
using jp.kshoji.unity.midi.util;

// 単一送信
MidiSend.To(deviceId).Channel(0).NoteOn(60, 100);

// 和音を連続送信
MidiSend.To(deviceId).Channel(0)
    .NoteOn(60, 127)
    .NoteOn(64, 127)
    .NoteOn(67, 127);

// 最初の出力デバイスへ CC 送信
MidiSend.ToFirstOutput().Channel(0).ControlChange(1, 64);
```

MidiManager との関係

`MidiSend` は `MidiManager` の薄いラッパーです。既存の `SendMidiNoteOn` 等と同等の動作をします。Unity エディタ上では、送信メッセージは [MIDI モニター](#) に OUT として表示されます。

詳細な送信 API については [MIDI 1.0 ランタイム \(MidiManager\)](#) の「MIDI 1.0 メッセージの送信」も参照してください。

PitchBendUtility

14-bit ピッチベンド用ヘルパー（センター = 8192）。Unity 非依存の pure C# です。

配置: Assets/MIDI/Scripts/Utilities/Messaging/PitchBendUtility.cs

メンバー	説明
Min / Max / Center	0 / 16383 / 8192
Clamp(amount)	0-16383 にクランプ
FromNormalized(normalized, zeroToOne)	float → 14-bit （ zeroToOne : 0-1、または両極 -1...+1）
ToNormalized(amount)	14-bit → 0-1 （センター ≈ 0.5）
Split / Combine	LSB/MSB（7+7 bit）の分解・合成

セミトーン範囲への変換は未実装です（[索引](#) の将来機能を参照）。

```
var amount = PitchBendUtility.FromNormalized(0.75f);
PitchBendUtility.Split(amount, out var lsb, out var msb);
MidiSend.To(deviceId).Channel(0).PitchWheel(amount);
```

MidiControlSmoothing

`MidiCcSmoother` やシーケンサ出力段で共有する EMA（指数移動平均）ヘルパーです。

配置: `Assets/MIDI/Scripts/Utilities/Messaging/MidiControlSmoothing.cs`

メソッド	説明
<code>ApplyEma(previous, raw, smoothFactor)</code>	EMA。 <code>smoothFactor</code> 0.01–1.0（小さいほど滑らか）
<code>ApplyEmaToMidi7(...)</code>	EMA 後に 0–127 へ丸め・クランプ
<code>ApplyEmaToMidi14(...)</code>	EMA 後に 0–16383 へ丸め・クランプ（ピッチベンド）

AutomationInterpolation

順序付きオートメーション・ブレイクポイント（正規化 0-1）をティック位置で評価します。

配置: `Assets/MIDI/Scripts/Utilities/Messaging/AutomationInterpolation.cs`

API	説明
<code>CurveKind.Linear</code> / <code>Smooth</code>	ポイント間の線形補間または smoothstep
<code>Evaluate(points, tick, curve)</code>	ティックでの値。先頭より前 / 末尾より後は端点、空リストは 0

MidiKeyboardLogic

エディタの仮想 MIDI コントローラー鍵盤と、ランタイム UI サーフェスの `MidiKeyboardElement` で共有する鍵盤レイアウトユーティリティです。

配置: `Assets/MIDI/Scripts/Editor/UI/PianoKeyboardElement.cs` (`MidiKeyboardLogic`)

API	説明
<code>IsBlackKey(note)</code>	黒鍵かどうか判定
<code>CountWhiteKeysBefore(note)</code>	指定ノートより前の白鍵数
<code>CountWhiteKeys(start, end)</code>	範囲内の白鍵数
<code>DefaultStartNote</code> / <code>DefaultEndNote</code>	デフォルト 2 オクターブ範囲 (C3-B4)

ランタイム UI の詳細は [ジャンルキット](#) を参照してください。

TempoMapExtractor

SMF（Sequence）からテンポ変更・拍子記号を時系列で抽出する静的ユーティリティです。SmfPlayer の再生時間計算や、譜面・タイムライン生成の基盤として使えます。

配置: Assets/MIDI/Scripts/Utilities/Smf/TempoMapExtractor.cs

API

メソッド	説明
Extract(Sequence)	Sequence からテンポマップを構築
Extract(byte[] smfData)	SMF バイナリから抽出
Extract(string filePath)	.mid ファイルから抽出

TempoMap

メンバー	説明
TempoChanges	テンポ変更（Tick / 秒 / BPM）
TimeSignatures	拍子記号
TotalDurationSeconds	総再生時間
GetBpmAt(tick)	指定 Tick の BPM
TickToSeconds(tick)	Tick → 秒
SecondsToTick(seconds)	秒 → Tick

詳細な使用例は [SMF ツール](#) を参照してください。同フォルダには次もあります:

ユーティリティ	役割
MeasureTimeUtility	小節長ティック（TicksPerBar / TicksPerBeat）、小節 ↔ 時間
TupletUtility	有理数連符のステップティック（例: 3:2）。index↔tick 可逆
SwingUtility	再生時のみの裏拍スイング遅延
MidiMetaMessageFactory	Tempo / Time Signature / Track Name メタの生成・読み取り

MidiScaleUtility

ノートが指定スケールに属するかを判定する静的ユーティリティです。Visual Scripting の **Is Note In Scale** ノードからも利用されます。

MidiScaleType

値	説明
Major	長音階
NaturalMinor	自然短音階
HarmonicMinor	和声短音階
MajorPentatonic	長調ペントニック
MinorPentatonic	短調ペントニック
Blues	ブルース
Dorian	ドリアン
Mixolydian	ミクソリディアン
Chromatic	クロマティック

API

メソッド	説明
IsNoteInScale(noteNumber, rootNote, scaleType)	ノートがスケールに含まれるか
GetIntervals(scaleType)	スケールの半音インターバル配列

```
using jp.kshoji.unity.midi.util;

// C メジャースケール（根音 C4=60）に D4(62) は含まれる
bool inScale = MidiScaleUtility.IsNoteInScale(62, 60, MidiScaleType.Major); // true
```

ScaleUtility / ChordRecognition

ScaleUtility は MidiScaleUtility を拡張し、複数ノートのスケール所属を一括判定します。

メソッド	説明
AllNotesInScale(notes, rootNote, scaleType)	全ノートがスケール内か
AnyNoteOutsideScale(notes, rootNote, scaleType)	いずれかがスケール外か
GetIntervals(scaleType)	MidiScaleUtility.GetIntervals のエイリアス

ChordRecognition.Recognize(activeNotes) は押下ノート集合から和音名（C, Cm7, Cmaj7 等）を推定します。三和音・四和音の主要パターンに対応しています。

```
var notes = noteTracker.GetActiveNotes(0);
var chord = ChordRecognition.Recognize(notes);
var inScale = ScaleUtility.AllNotesInScale(notes, rootNote: 60, MidiScaleType.Major);
```

詳細なリアルタイム検出は [ゲームプレイ向けコンポーネント](#) の MidiChordDetector を参照してください。

関連ドキュメント

- [MIDI 1.0 ランタイム \(MidiManager\)](#)
- [SMF ツール \(SmfPlayer / MidiRecorder\)](#)
- [エディタツール \(MIDI モニター\)](#)
- [スタートガイド](#)